

Slam- Video Navigation - Exercise 2

Students: Ron Keter, Yehonatan Hirschel

Git Repository: github.com/yehonatan1305/SLAM-VN

May 6, 2025

Question 1

it is given that two pair of images are rectified, meaning the images "matched epipolar projections". These are projections in which the epipolar lines run parallel with the x-axis and match up between views, and consequently disparities between the images are in the x-direction only, therefore there is no y disparity. so for a pair (p_1, p_2) . points the left and right images plane correspondingly. for them to be a match they essentially should have no y disparity. So the cause for this pattern is the fact that stereo image are rectified and it stems from the constraints of epipolar geometry we have seen in class.

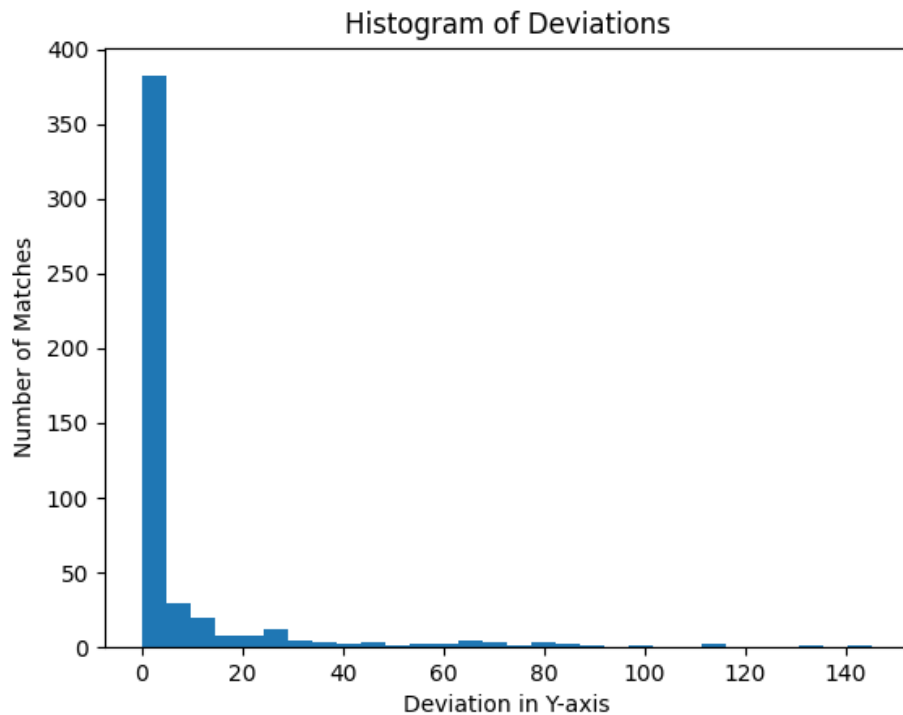


Figure 1: Histogram of deviations (ex2.py lines 80-85)

Percentage of matches deviating by more than 2 pixels: 35.20%

Question 2

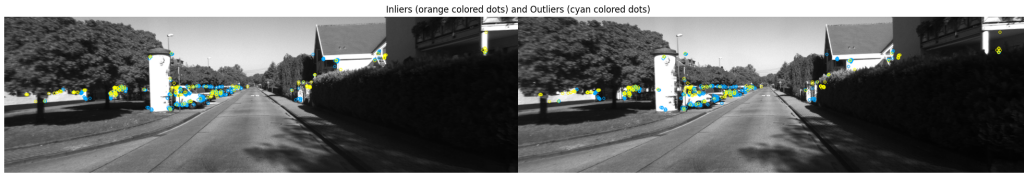


Figure 2: Accepted matches (inliers) in orange and rejected matches (outliers) in cyan (ex2.py line 99)

- Number of rejected matches by the pattern: 176.
- Our rejection policy limits the two features to have the same y-th coordinate, therefore the two keypoints should share the same row ± 2 pixels. Let us assume that the image is divided into N rows and M columns, and that the erroneous matches are uniformly distributed across the image. For each image, the probability of a match being in the same row is $1/N$ and the probability of a match being in the same row ± 2 pixels is $4/N$ (if we use continuous subpixel numbering or $5/N$ if we use discrete pixel numbering). Therefore, the probability of an erroneous match to be rejected is $1-(4/N)$ by this rejection policy.
- The assumption of uniform distribution of erroneous matches is not realistic. In real-world scenarios, like in our street scenes, the image can be divided into distinct semantic regions-such as the road at the bottom, buildings and trees higher up, and cars somewhere in between. Because features tend to occur within these semantically consistent regions, incorrect matches are more likely to share similar vertical positions (y-coordinates). As a result, erroneous matches often fall within the same general area, violating the uniform distribution assumption. For this reason the probability of an erroneous match to be rejected is lower than the one we calculated above (with the uniform distribution assumption), meaning that the actual number of accepted erroneous matches is likely to be higher.

Question 3

For visualization purposes, we have done the following steps:

1. We inverted the y axis.
2. we crop the plot using axis limitations.
3. we used axis equals.

3D Points from Linear Least Squares

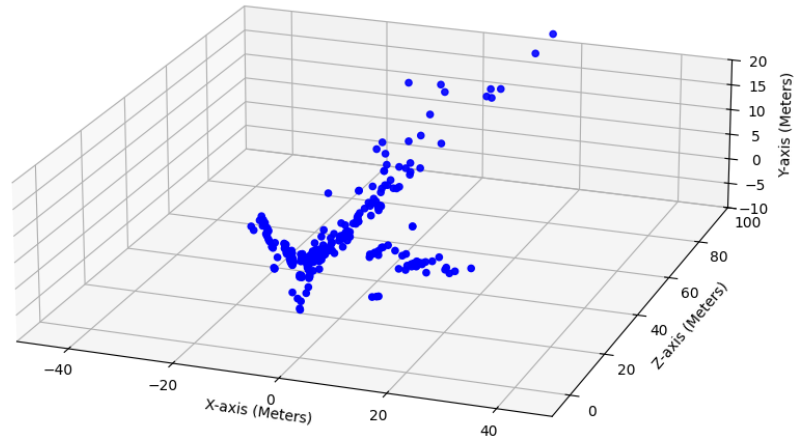


Figure 3: 3D points calculated using Linear Least Squares triangulation (ex2.py lines 69)

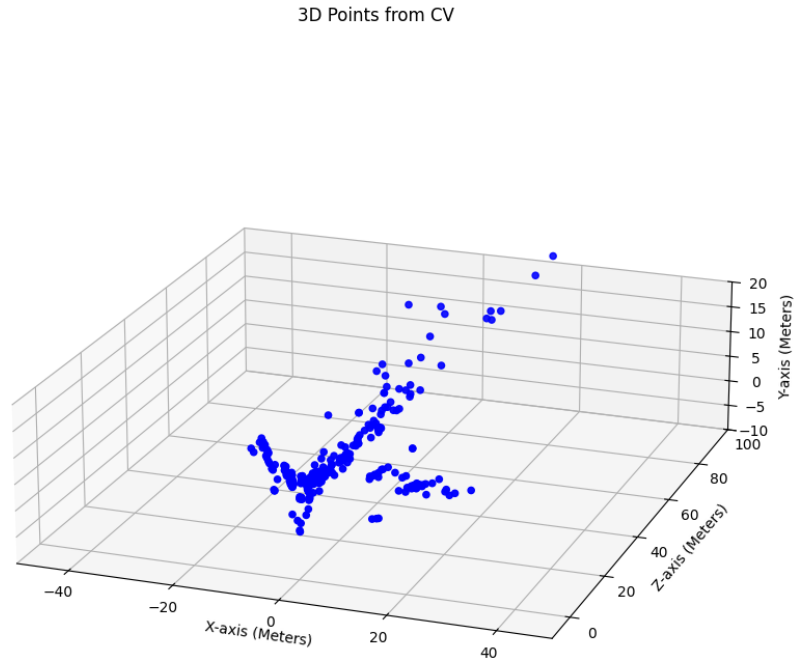


Figure 4: 3D points calculated using OpenCV's `cv2.triangulatePoints` (ex2.py line 58)

Question 4

We observed that some 3D points have negative Z values, meaning they appear behind the camera, which is clearly incorrect. The most likely cause of this issue is incorrect matching between keypoints. This could result from true mismatches or from small localization errors during keypoint detection-such as when a keypoint is placed slightly to the left or right of the actual feature. We can remove these outliers based on their negative Z values, or even detect them before triangulation. Specifically, if the x -coordinate of a keypoint in the right image is greater than that of the corresponding keypoint in the left image the resulting triangulated point is likely to fall behind the camera, resulting in a negative Z value.

3D Points from 1608

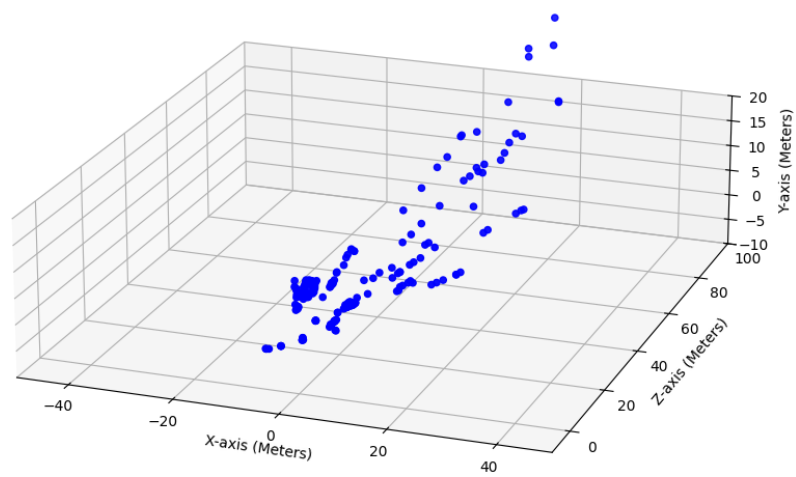


Figure 5: Filtered 3D points for sample 1608 (ex2.py line 132)

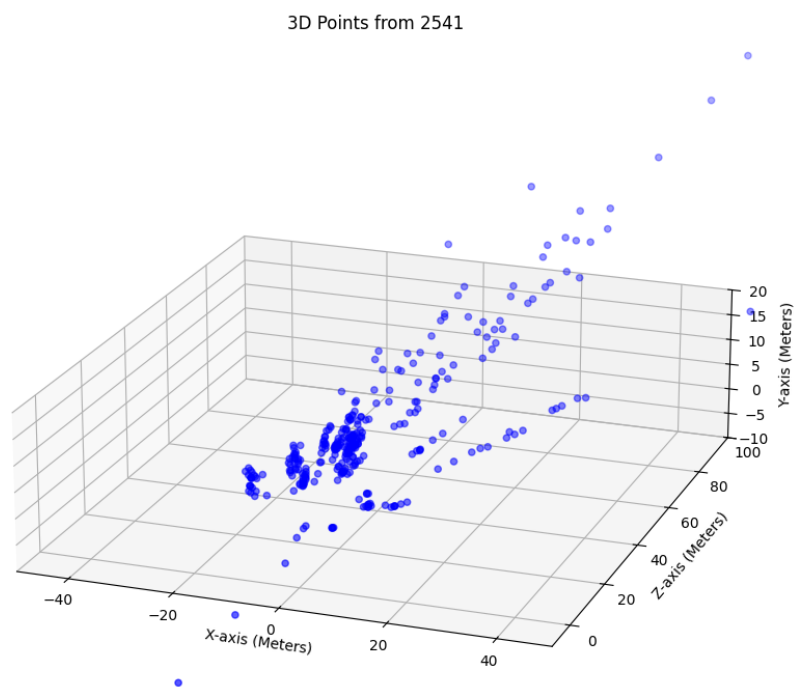


Figure 6: Filtered 3D points for sample 2541 (ex2.py line 132)

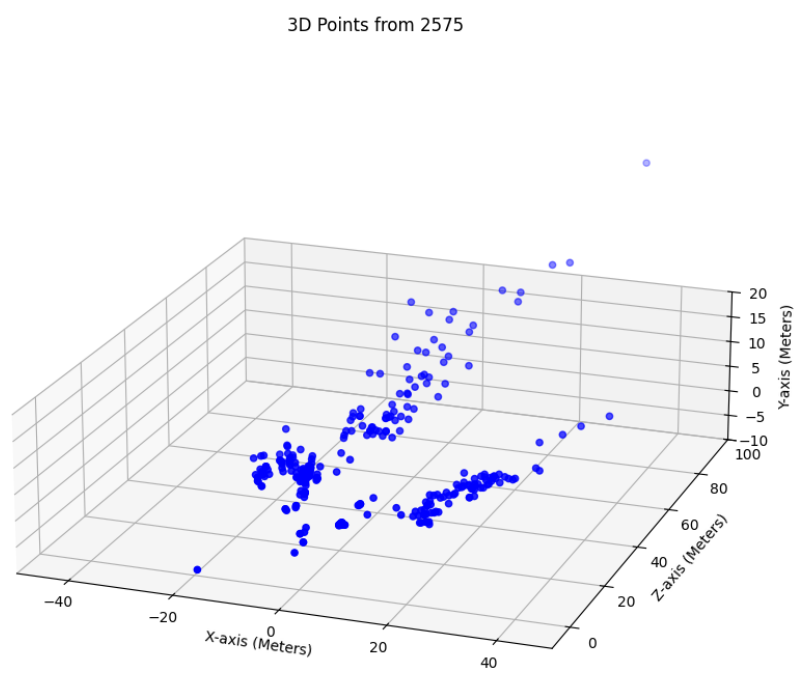


Figure 7: Filtered 3D points for sample 2575 (ex2.py line 132)